Pegasus: Probabilistic Performance Guarantees for Agentic Provisioning of Software Services

Boris Sedlak, *Member, IEEE*, Milos Gravara, *Member, IEEE*, Víctor Casamayor Pujol, *Member, IEEE*, Stefan Nastic, *Member, IEEE*, Schahram Dustdar, *Fellow, IEEE*

***Abstract*—**

***Index Terms*—Performance Guarantees, Service Quality, Gaussian Processes, Performance Modeling, Service Composition**

I. Introduction

Large Language Models (LLMs) have risen as universal interpretation and generation engines for text-based contents, introducing a major disruption for software development [7]. Traditionally, computing services were developed manually according to stakeholder requirements, and operated in close-loop controls to continuously meet operational thresholds. Within that context, LLMs appeared as a fast and cost-efficient enabler for autonomous software development and deployment, opening new branches of service research [2]: automatic generation [6], [15] and recomposition [24] of software services. The combined vision from these works is a *future internet*, where agents aim to continuously satisfy high-level user intents by generating, composing, and reconfiguring software applications and their services [1].

Traditional service-oriented architectures already provide clear roles and interfaces to operate such systems [25]: the service provider and the service consumer establish Service Level Agreements (SLAs) that specify the expected quality through measurable Service Level Objectives (SLOs). In this future internet, applications will be tailored to individual clients and their SLOs [27]. However, creating this vast space of possible application also means that potential compositions or configurations will often lack end-to-end benchmarks *prior to deployment*. It is neither possible nor cost-efficient to benchmark this search space exhaustively before offering a solution to a consumer; hence, service providers require probabilistic models for providing guarantees about the quality of different service compositions and configurations before deployment.

Applications themselves can be recomposed into systems of systems [23]; to decrease the overall complexity, individual systems (or services) must provide clear interfaces and guarantees. So suppose we need to ensure that one system's latency is below a threshold, it is not enough to *estimate* that the tail latency (e.g., the P_{99} percentile) will be below the threshold, but the application must *guarantee* this with a certain probability. To avoid SLA violations, providers could simply give increasingly conservative performance estimates, which could only be fulfilled by provisioning more resources, increasing the price for the consumer, and wasting the provider's resources. The challenge is thus to make the performance guarantee as *tight* as possible. For a service composition S, also called an application, with topology $\{s_1 \rightarrow s_2 \rightarrow \dots\}$, a provider needs to be able to accurately answer the following questions:

- What performance could we guaranteed for this application, before conducting any end-to-end benchmark?
- What performance could we guarantee if we replaced a service (e.g., s_1) with another one (e.g., s_x) that was not benchmarked end-to-end in this exact application?
- What performance could we guarantee if we reconfigured an individual service (e.g., s_1) to a configuration that was not benchmarked end-to-end in this application?

While composition and configuration are operationally decoupled—services can be reconfigured without altering the topology, and vice versa—they are logically entangled: estimating composite performance requires understanding individual service dynamics under limited empirical observations. Optimization techniques like branch-and-bound can reduce the candidate space, but evaluating candidate solutions remains the core bottleneck. Learning-based optimizers must contend with varying degrees of empirical evidence, ranging from well-observed service instances to completely unseen configurations and compositions.

TODO: Add summary paragraph introducing the contribution/solution: State the gap in existing work (e.g., deterministic code contracts vs. black-box microservices) and summarize the proposed PPG framework combining SVGPs and Stochastic Network Calculus.

II. Preliminaries

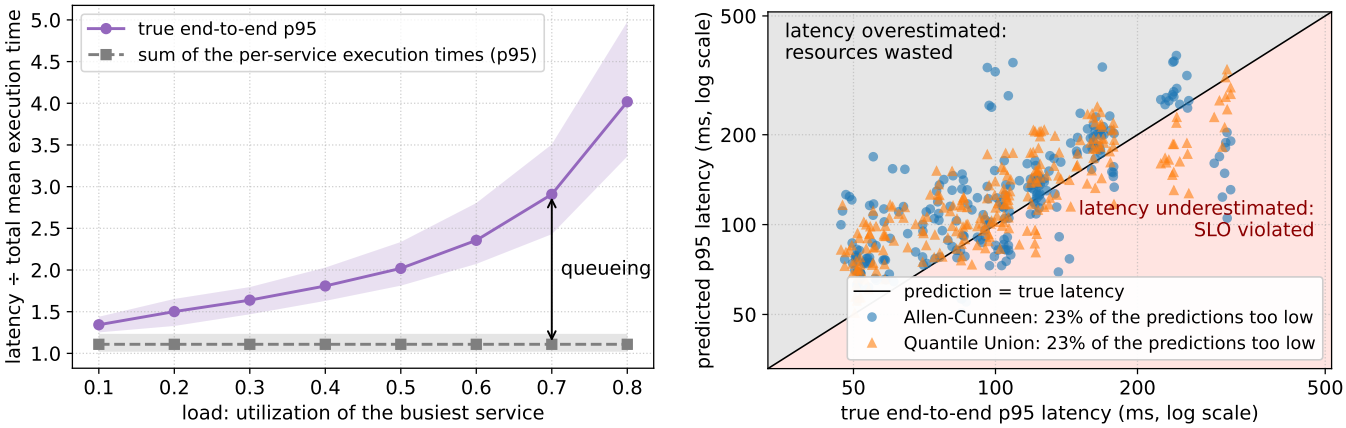
A. Motivating Example

Consider a customer that requests a real-time drone monitoring application, which can be structured as a four-stage pipeline: (1) video stream ingestion from the drone, (2) object detection along the trajectory, (3) trajectory adjustment calculation, and (4) telemetry reporting to an external traffic planner. Logically, this application is composed of multiple connected microservices $S = \{s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4\}$.

Each service in this application can be configured in terms of allocated hardware resources (e.g., CPU cores), the number

B. Sedlak, M. Gravara, S. Nastic, and S. Dustdar are with the Distributed Systems Group, TU Wien, Vienna, Austria (e-mail: b.sedlak@dsg.tuwien.ac.at; m.gravara@dsg.tuwien.ac.at; s.nastic@dsg.tuwien.ac.at; dustdar@dsg.tuwien.ac.at).

V. Casamayor Pujol is with the Department of Engineering, Universitat Pompeu Fabra, Barcelona, Spain and S. Dustdar is with ICREA (e-mail: v.casamayor@upf.edu; schahram.dustdar@upf.edu).



(a) Per-service execution times do not add up to the end-to-end tail latency.

(b) An accurate prediction is not a guarantee and can err in both directions.

Fig. 1. Composing per-service performance models does not yield an end-to-end guarantee (computer vision application under Poisson arrivals from Section V). **(a)** True end-to-end P_{95} and the sum of the per-service P_{95} execution times, both divided by the total mean execution time of the chain, against the utilization of the busiest service; lines show the median and bands the interquartile range over the configurations. **(b)** Predicted versus true P_{95} at utilization 0.5 (5 training sets, 240 predictions per method) for a queueing formula on learned execution times (*Allen-Cunneen* [4]) and learned per-service latency quantiles (*Quantile Union*); the legend gives the share of predictions below the true latency.

of service replicas, and internal service parameters (e.g., object detection model resolution or batch size). While all these configuration knobs influence execution time, the customer is solely interested in end-to-end *performance* (e.g., total latency ≤ 100 ms) and overall *cost* (e.g., minimize resources).

Furthermore, external systems like the traffic planner pose strict workload bounds, e.g., updates must arrive consistently every 100 ms. Adhering to these bounds in distributed systems typically relies on reporting high-percentile tail latencies (e.g., P_{90} or P_{99}). However, the specified application topology and configuration have not been executed and benchmarked end-to-end so far; the provider only has measurements of the individual services, collected in isolation or in another existing applications. The following section shows why these measurements alone do not yield the required guarantee.

B. Research Questions

Figure 1 shows why naive analysis and composition of service observations fail to guarantee performance, and motivates the two research questions that this paper addresses. The application is a chain of three computer vision services from our testbed in Section V, each configurable in its number of service replicas and quality knobs (e.g., input resolution, model size). For each of the 50 measured configurations of the chain, we replay the per-item execution times measured at every service through FIFO queues, with Poisson arrivals at a rate that keeps the busiest service busy for a given share of the time (its utilization). All models are trained only on runs of the services alone or in other chains, never on this chain.

Composing Tail Latencies under Load. The end-to-end latency of an item is the sum of its execution times *plus* the time it waits in the queue of every service. This waiting time cannot be observed at an individual service: it depends on how irregularly items enter the application (the external arrival process), on how each service releases items to the next one (the internal composition), and on how close each service runs to its capacity (the load). Figure 1a shows that the sum of the per-service P_{95} execution times underestimates the end-to-end P_{95} at every load, and that the gap grows significantly with increasing load. Benchmarking a candidate composition can capture end-to-end latency, including queuing time [21], but, as argued in the introduction, the space of possible compositions and configurations is too large to benchmark exhaustively. **Question #1:** The open research question is thus how to derive the end-to-end tail latency of a composition, including its queueing time, purely from observations of individual services and a workload description.

Guarantees for Unseen Configurations. An SLO requires a latency bound that is exceeded with at most a given probability; however, performance models are fitted to be accurate [8], [30], i.e., *close* to the measured value on average, but not necessarily above it. For a configuration that was never observed, such a prediction can err in either direction: an overestimated latency wastes resources, an underestimated one violates SLOs. Figure 1b shows that two predictors that account for queueing, trained only on observations of individual services, overestimate the latency for most unseen configurations but underestimate it for a sizable share. Even quantile predictors [14] give no indication of whether a configuration is well covered by observations or far away from all of them. **Question #2:** The open research question is therefore how to turn limited evidence about each service into an end-to-end bound that holds with a stated probability, and becomes more conservative where evidence is missing.

C. Background and Related Work

TODO: To improve the flow, we must split this section: background stays here, and RW goes to the paper's end

We have grouped related work by how it obtains the performance of an application: (1) analytical models derive it from a formal description of the system, (2) learned models fit it to

measurements of the entire application, and (3) compositional models assemble it from models of the individual services. For each group, we explain *how* it models performance and *why* it leaves the presented research challenges open.

1) Analytical Performance Models: Queueing theory is a classical tool for relating workload and capacity to response time [16]. For instance, Gandhi et al. [13] model the performance of individual servers as M/G/1 queues[1], and use this model for finding a power allocation that minimizes overall response time. Layered Queueing Networks [10] extend queueing networks to software services that hold one resource while waiting for another, as in nested client–server calls. Both target mean response times in steady state rather than tail probabilities (Challenge #2). Network calculus bounds delays instead of averaging them: Duan [5] describes network and compute services through latency–rate capability profiles, merges these profiles into one end-to-end profile, and derives the maximum delay of the composite service. This bound is deterministic, i.e., it covers the worst case and thus becomes overly conservative for services with variable execution times; Stochastic Network Calculus (SNC) [9], on which we build, instead bounds the *probability* of exceeding a given delay. SNC assumes that the MGFs of arrivals and service exist; an extension to heavy-tailed traffic replaces them by power-law envelopes at the price of considerably looser bounds [22].

Closest in spirit, Iyer et al. [17] compute *performance contracts* for software network functions without executing them: symbolic analysis of the source code yields conservative upper bounds on the memory accesses and execution cycles per packet, expressed as functions of workload-dependent variables; contracts for chains of network functions are computed recursively. As with worst-case execution time analysis [32], this requires access to the source code, whereas microservices are typically deployed as opaque containers. Moreover, these contracts bound the processing effort per packet, not waiting time that accumulates under load (Challenge #1).

Common to all analytical models is that their parameters (e.g., service rates, branching probabilities, or code paths) must be supplied for the exact system under study. None of them expresses how reliable a prediction is for a configuration that was never measured (Challenge #2).

2) Learning Models of Entire Applications: A second line of work treats the application as one big black box and learns its end-to-end performance from empirical measurements; we call this the *top-down* approach. *Ernest* [30] fits a small parametric model to short profiling runs of an analytics job on samples of the input data, and extrapolates the running time to larger datasets and clusters. *CherryPick* [3] replaces the fixed model structure with a Gaussian Process (GP) and uses its confidence intervals to choose the next cloud configuration to benchmark; the search stops as soon as the model can identify the best configuration. *Microscaler* [35] similarly combines Bayesian optimization (BO) with a heuristic to find the number of instances at which a microservice meets its SLA. Production systems are more conservative: Google's *Autopilot* [26] sets resource limits from moving-window statistics of past usage plus a safety margin, optionally choosing among many such window models by their historical cost.

Top-down models are simple in the sense that they require no knowledge of the application's internal structure: a single model maps the configuration to the end-to-end performance. This is also their limitation: replacing a service or changing the topology changes the modeled function itself, so the end-to-end performance must be benchmarked again (Challenge #1). Furthermore, where uncertainty is modeled at all, as in *CherryPick*, it steers the search towards a good configuration but is not converted into a latency guarantee (Challenge #2).

3) Compositional Models: The *bottom-up* alternative predicts the quality of a composition from its constituent services, which has a long tradition in service-oriented computing. Gallotti et al. [12] translate activity diagrams into stochastic models for a probabilistic model checker; the execution time and reliability of each service are annotations supplied at design time. Wu et al. [34] use a hidden Markov model to predict whether a composition of Cloud services will satisfy the requested QoS, and Fanjiang et al. [8] evolve time-series predictors that forecast the QoS of web services. Geebelen et al. [14] come closest to a guarantee: a Petri-net simulation merges the monitored response times of the elementary services into a time series of the composition, and a kernel-based quantile estimator with an online offset correction predicts the quantile named in the SLO, such that the violation frequency converges to the agreed level. None of these works, however, learns QoS as a function of resource allocation, service configuration, and arrival rate. They can assess compositions of services as currently deployed, but not a composition that involves a configuration without observations; likewise, the guarantee of Geebelen et al. holds asymptotically in the number of observations (Challenge #2).

Fluxion [21] is the closest learned counterpart to our work. It trains one model per service (e.g., a deep neural network or a GP) that maps the service's configuration knobs, its observed states, and the latency percentiles of its downstream services to a latency percentile of the service itself. These models are chained into an inference graph that mirrors the service dependencies; after a service is scaled or replaced, only the model of that service is retrained. However, the inference graph propagates point estimates: Fluxion is evaluated by its mean absolute error, and its authors note that prediction errors accumulate along the graph in a way that is not trivial to derive analytically. A predicted P_{90} can therefore err in either direction, and the output does not reveal any uncertainty about whether a service model was queried at a configuration it has never seen (Challenge #2).

D. Summary

In summary, analytical models compose rigorously but need access to parameters (e.g., source code); top-down approaches can wrap around black-box applications, but this prevents

[1]Note that M implies a *Markovian* (or Poisson) arrival process, G a *General* service time distribution, i.e., described through a mean and variance, and $c = 1$ indicates a single server. *Pegasus* supports a wider range of arrival processes and multi-server queues with $c \geq 1$, but assumes a sub-Gaussian service process (e.g., Gaussian or bounded execution times).

generalization to unseen topologies; and existing compositional models propagate predictions without any degree of uncertainty. *Pegasus* (our approach) combines the strengths of these approaches: it learns the execution time distribution of each black-box service from local observations, including the epistemic uncertainty for unseen configurations, and composes per-service models into a probabilistic bound on the end-to-end latency for unseen service compositions.

III. Problem Definition

A. System Model

We model a service composition S, also called an application, as a Directed Acyclic Graph (DAG) with available microservices (vertices) $V = \{s_1, s_2, \ldots, s_n\}$, where directed edges $E \subseteq V \times V$ represent the paths along which items (e.g., requests or data frames) are passed on. For a sequential pipeline, this simplifies to $S = \{s_1 \to s_2 \to \cdots \to s_n\}$ with $n = |S|$ services.

Services. Each service $s_i \in V$ consists of c_i instances, where all instances share one configuration $x_i = \langle r_i, \kappa_i \rangle$, where

- $r_i \in \mathcal{R}_i$ are the global resource allocations (e.g., CPU quota, memory limit) across all service instances,
- $\kappa_i \in \mathcal{K}_i$ are its software configuration knobs (e.g., model size, data quality) unified across all instances, and
- $c_i \in \mathbb{N}$ is the number of servers/workers for s_i.

A service processes up to c_i items at a time, one per worker, in the order of arrival; items that arrive while the service is busy wait in its queue. The *execution time* Y_i of s_i is the time (in seconds) from the start to the end of processing one item. Due to runtime variability (e.g., thread scheduling, input size), Y_i is a random variable whose unknown distribution $P_{Y_i|x_i}$ depends on the configuration of the service. The *latency* of an item at s_i is its waiting time in the queue plus its execution time, $W_i = Q_i + Y_i$.

For each service, empirical evidence is available only as a sparse dataset of execution times observed at N_i configurations, $\mathcal{D}_i = \{(x_i^{(j)}, y_i^{(j)})\}_{j=1}^{N_i}$, collected while the service ran in isolation or within other compositions. This means that large regions of the configuration space $\mathcal{R}_i \times \mathcal{K}_i$ remain unobserved; likewise, only a small share of all possible service compositions $(V \times V)^n$ have been benchmarked end-to-end.

Workload. Items enter the composition at its first service according to an *external arrival process*, which we describe by the gap T (in seconds) between two consecutive arrivals. We define the mean arrival rate λ (items per second) as

$$\lambda = \frac{1}{\mathbb{E}[T]}, \tag{1}$$

and the variability of the arrivals C_a^2 through the squared coefficient of variation as

$$C_a^2 = \frac{\mathrm{Var}(T)}{\mathbb{E}[T]^2}. \tag{2}$$

An item that leaves a service immediately enters the next one, so every downstream service receives the departures of its predecessor.

TODO: Place the utilization formula somewhere

$$\rho_i = \frac{\lambda \times m_i}{c_i} \tag{3}$$

Probabilistic Performance Guarantees. Let D_S denote the end-to-end latency in a composition S under the configuration vector $\mathbf{x} = (x_1, \ldots, x_K)$, i.e., the time an item spends from its external arrival until it leaves the last service. Given a maximum permissible violation probability $\epsilon \in (0, 1)$ specified in an SLO, a latency bound d_S must satisfy

$$\mathbb{P}\left(D_S > d_S\right) \leq \epsilon, \tag{4}$$

which means that d_S is an upper bound on the $(1-\epsilon)$ quantile of the end-to-end latency.

TODO: Might need an example with some values

Because the execution-time distributions are modeled using sparse observations $\mathcal{D}$, the latency bound in Eq. (4) contains noise and uncertainty around poorly explored configurations. A *Probabilistic Performance Guarantee* (PPG) therefore has two parameters: the violation probability ϵ, which covers the variability between items, and a confidence level $\gamma \in (0, 1)$, which covers the uncertainty of the learned models. A PPG is a latency bound $d_S(\mathbf{x}, \epsilon, \gamma)$ for which Eq. (4) holds with confidence γ.

B. Problem Statement.

Given a composition S, a configuration vector $\mathbf{x}$, an external arrival process λ, C_a^2, a violation probability ϵ, and a confidence level γ, derive the PPG $d_S(\mathbf{x}, \epsilon, \gamma)$ in a *zero-shot manner*, which means purely from the per-service datasets $\{\mathcal{D}_1, \ldots, \mathcal{D}_K\}$ and without any end-to-end measurement of S. The bound should be as tight as possible, since every unit of slack translates into over-provisioned resources, and account for the queuing at every service.

C. Foundational Assumptions

To obtain performance bounds for black-box service compositions without end-to-end measurements, our framework makes the following assumptions about the domain:

1) *Performance Isolation:* Services run in isolated execution environments (e.g., containers with strict CPU and memory limits), and the CPU quota per service suffices for executing all its workers without throttling. The execution time of a service is therefore neither affected by co-located services nor by its own load.
2) *Multi-Server FIFO Services:* Every service has c_i workers that share one unbounded queue and process items in the order of their arrival (i.e., FIFO). For constructing the bound, we assume that a worker receives every c_i-th item (*cyclic assignment*), turning it into a single-server queue whose gap between two items is the sum of c_i consecutive inter-arrival times. Cyclic assignment can only increase the waiting time [33]; hence, upper bounds under cyclic assignment also hold for shared queues.
3) *Load-Independent Execution Times:* The distribution of the execution time Y_i depends only on the configuration x_i; thus, Y_i depends neither on the arrival process

nor on the services before s_i, and the execution times of different items at s_i are independent. This allows execution times observed in isolation, or in a different composition, to be reused for unseen compositions.

4) *Sub-Gaussian Execution Times:* The moment-generating function (MGF) of the execution time Y_i is bounded by that of a Gaussian with mean $m_i = \mathbb{E}[Y_i]$ and variance proxy σ_i^2, both depending on x_i, for all $\theta > 0$:

$$M_{Y_i}(\theta) = \mathbb{E}\left[e^{\theta Y_i}\right] \leq \exp\left(\theta\, m_i + \frac{\theta^2}{2}\,\sigma_i^2\right). \quad (5)$$

This holds, e.g., for Gaussian and for bounded execution times; heavier tails are discussed in Section VI-A.

5) *Renewal Arrivals:* The gaps T between external arrivals are independent, identically distributed (i.i.d.), and independent of the execution times, and their Laplace transform $\mathbb{E}[e^{-\theta T}]$ is known. This covers, for instance, Poisson arrivals, periodic arrivals with jitter, and gamma-distributed gaps of any C_a^2.

6) *Arrivals Inside the Composition:* A service that is not the first of the composition is bounded as if it received the external arrival process, refined by the minimum spacing that its predecessor enforces between two departures (Section IV-B). This is conservative when the external arrivals are at least as variable as the execution times, as with Poisson arrivals; for nearly periodic arrivals combined with highly variable execution times and high utilization, the departures of a service can be burstier than the external arrivals.

Independence between the execution times of an item at *different* services is not assumed in general; it is only required by one of the two presented framework variants (Section IV-C).

IV. Pegasus Framework

Given limited observations from individual services, our framework, called *Pegasus*, allows bounding the performance of any potential composition and configuration of these services. This bound—the PPG—presents a probabilistic performance guarantee for the end-to-end latency according to an external arrival process and a desired confidence level. Our methodology consists of three steps, as shown in Figure 2: (1) learning a probabilistic model of the execution time of each service, (2) deriving a latency bound for each service from its execution time model and expected arrival process, and (3) composing the service bounds into a performance guarantee for a complete application. Only the first step requires training data for modeling the performance of individual services; the arrival processes and the composition enter analytically in the second and third step. This separation makes the guarantee available for compositions that were never benchmarked.

In the following subsections, we explain the three steps of the *Pegasus* framework in more detail. For each step, we also provide technical details on how Pegasus was implemented. The implementation is publicly accessible on the Pegasus Repository; Appendix X contains an extended description of the repository and how to use Pegasus for a custom dataset.

A. Learning per Service Performance Models

The first step trains, for each service s_i, a probabilistic model that predicts the execution time Y_i of an item from the service configuration x_i. Two constraints apply to this step: (1) the scaling behavior of services is rarely linear [3], e.g., performance saturates after providing sufficient RAM, and (2) services live in containers, whose source code can rarely be analyzed to make assumptions about their functional behavior. Gaussian Processes (GPs) satisfy both constraints, as they are non-parametric and do not impose a structure on the learned function. Unlike point-estimate models (e.g., neural networks), a GP also quantifies the uncertainty of its prediction, which protects the bound at unobserved configurations.

We train one GP per service on its dataset $\mathcal{D}_i$. For a configuration x_i, which may be unobserved, the GP provides three quantities:

- the predicted mean execution time $m_i(x_i)$;
- the *epistemic* standard deviation $\sigma_{\mathrm{ep},i}(x_i)$, i.e., the uncertainty of the model about the mean, which is large far from observed configurations and shrinks with more data; and
- the *aleatoric* standard deviation $\sigma_{\mathrm{al},i}(x_i)$, i.e., the scatter of individual items around their mean, a property of the service that does not shrink with more data.

The following two steps treat these two kinds of uncertainty differently. In the remainder of this section, we omit the argument x_i where it is clear from the context.

Implementation. Each service model is an exact GP (scikit-learn) with a scaled Matérn-5/2 kernel, one length scale per input, and an additive white-noise term, fitted on standardized inputs by maximizing the marginal likelihood (five restarts). $\sigma^2_{\mathrm{ep},i}$ is the posterior variance of the latent function and $\sigma^2_{\mathrm{al},i}$ the fitted noise level. The number of workers c_i is not a model input, since it does not change the execution time (Assumptions 1 and 3). Exact inference costs $\mathcal{O}(N_i^3)$; for large datasets, a sparse GP can replace it without changing the following steps. On the testbed, a benchmark run processes many items at one configuration, so we train on two statistics per run: the mean execution time, which is far less noisy than a single item, and the spread $(95\text{th percentile} - \text{median})/1.645$, i.e., the standard deviation of a Gaussian with that median and 95th percentile. A second GP per service predicts the spread and provides $\sigma_{\mathrm{al},i}$; $\sigma_{\mathrm{ep},i}$ is then the predictive standard deviation of a new run mean, which also contains the scatter between repeated runs. Both GPs are fitted on the logarithm of their target, because knobs act multiplicatively on the execution time (e.g., a larger model at a higher resolution), and their length scales are bounded below by 0.3 to prevent a collapse to a GP that only interpolates the training runs.

B. Bounding Latency per Service

The second step turns the model of a service into a bound on its latency $W_i = Q_i + Y_i$ under a given workload: a value $d_i(\epsilon)$ that the latency of an item exceeds with probability at most ϵ. Unlike the execution time, the latency includes the waiting time Q_i in the queue of the service, and therefore

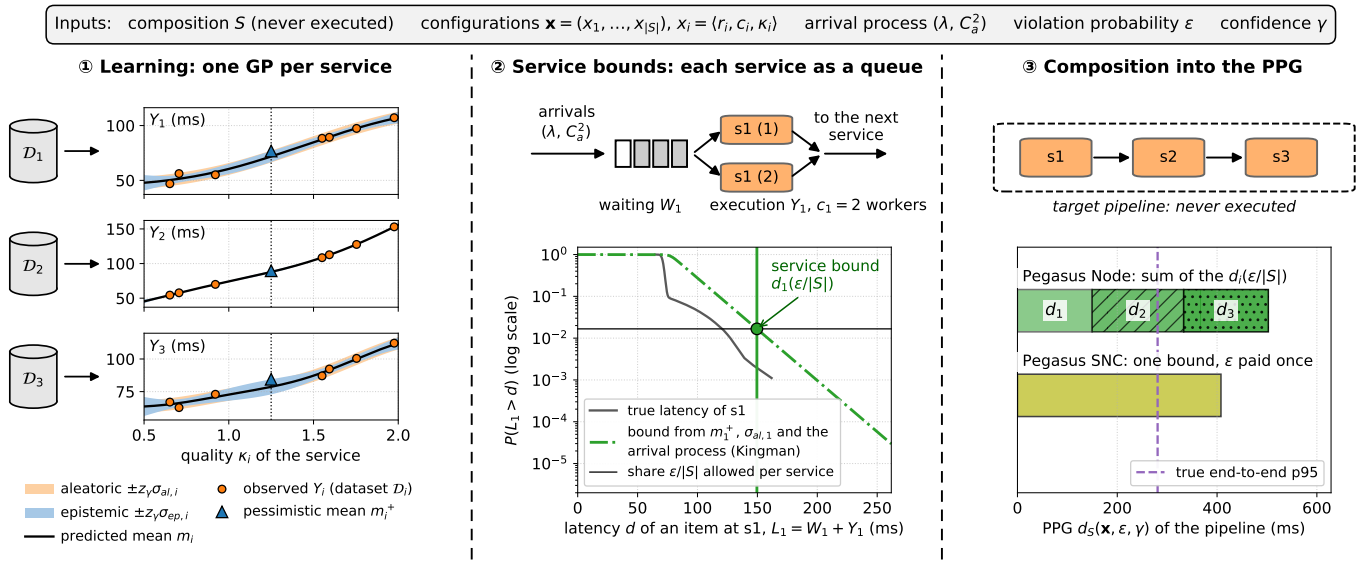


Fig. 2. Pegasus for a application that has never been executed. ❶ One GP per service, trained only on that service's execution times $\mathcal{D}_i$, yields a pessimistic mean m_i^+ that grows with the epistemic uncertainty; ❷ Each service is bounded as a FIFO queue with c_i workers, which gives the service bound $d_i(\epsilon/K)$; ❸ The service bounds are composed into the PPG $d_S(\mathbf{x}, \epsilon, \gamma)$, either as their sum (Pegasus Node) or as one application-wide bound (Pegasus SNC).

depends on how many items arrive. Both this step and the composition operate on MGFs: Chernoff's bound $\mathbb{P}(Y > d) \leq e^{-\theta d} M_Y(\theta)$, $\theta > 0$, controls the tail of a time Y, the MGF of a sum of independent times is the product of their MGFs, and the Laplace transform of the gaps (Assumption 5) is their MGF at $-\theta$.

Pessimistic Execution-Time Model. The aleatoric scatter is independent between items, so it partly averages out over a busy period. The epistemic error does not: if the model underestimates the mean by 5 ms, *every* item takes 5 ms longer, and a busy period of 100 items grows by 500 ms. Adding both variances per item would therefore underestimate the queue. Instead, we let the epistemic uncertainty shift the mean and keep only the aleatoric variance per item:

$$m_i^+ = m_i + z_\gamma \, \sigma_{\mathrm{ep},i}, \qquad Y_i \sim \mathcal{N}\big(m_i^+, \, \sigma_{\mathrm{al},i}^2\big) \,, \tag{6}$$

where z_γ is the γ-quantile of the standard normal distribution (e.g., $z_\gamma \approx 1.64$ for $\gamma = 0.95$). According to the model, the true mean execution time lies below the *pessimistic mean* m_i^+ with probability γ; this is where the confidence level of the PPG enters. With Eq. (5), the MGF of the execution time is $M_i(\theta) = \exp(\theta m_i^+ + \frac{\theta^2}{2}\sigma_{\mathrm{al},i}^2)$.

Waiting Time. The waiting time of an item at a FIFO service is the largest backlog of work that the items before it left behind. Under cyclic assignment (Assumption 2), each worker is a single-server queue whose gap between two items is the sum of c_i external gaps T. Since the gaps are independent (Assumption 5), Kingman's martingale bound [19] states that this backlog has an exponentially decaying tail,

$$\mathbb{P}(Q_i > w) \leq e^{-\theta_i^* w}, \text{ with} \tag{7}$$

$$\theta_i^* = \sup\big\{\theta > 0 : M_i(\theta)\, \mathbb{E}\big[e^{-\theta T}\big]^{c_i} \leq 1\big\}. \tag{8}$$

The *decay rate* θ_i^* (in 1/s) decreases with everything that makes a queue long: a larger mean or variance of the execution time, a higher arrival rate, and more variable arrivals. For Poisson arrivals at rate λ, for instance, $\mathbb{E}[e^{-\theta T}] = \lambda/(\lambda+\theta)$. If no $\theta > 0$ satisfies the condition, the service is overloaded and has no finite bound; since the condition uses m_i^+, this can also happen for a stable service whose mean is too uncertain.

Latency. The waiting time of an item depends only on the items before it, and is therefore independent of its own execution time. Combining Eq. (7) with the Gaussian execution time of Eq. (6) gives the tail of the latency in closed form,[2]

$$\mathbb{P}\,(W_i > d) \leq \bar{\Phi}\left(\frac{d - m_i^+}{\sigma_{\mathrm{al},i}}\right) + e^{-\theta_i^*(d - m_i^+) + \frac{1}{2}(\theta_i^* \sigma_{\mathrm{al},i})^2} \, \Phi\left(\frac{d - m_i^+ - \theta_i^* \sigma_{\mathrm{al},i}^2}{\sigma_{\mathrm{al},i}}\right), \tag{9}$$

where Φ is the standard normal distribution function and $\bar{\Phi} = 1 - \Phi$. The first term is the probability that the execution time alone exceeds d; the second term adds the waiting time. The *service bound* $d_i(\epsilon)$ is the smallest d for which the right-hand side is at most ϵ.

Services Inside an Application. Only frontend services receive the external arrivals; every subsequent service s_i receives the departures of its predecessor s_{i-1}. We bound s_i with the external arrivals (Assumption 6), and tighten the bound if s_{i-1} has a single worker: its departures are then at least one execution time of s_{i-1} apart, so a worker of s_i, which receives every c_i-th item, sees gaps of at least c_i such execution times. If s_{i-1} is slower than all c_i workers of s_i together, s_i hardly builds up a queue, whatever the external arrival rate. Applying

[2]Eq. (9) is the only step that uses the Gaussian shape of Y_i; all other steps only need the sub-Gaussian bound of Eq. (5). For a sub-Gaussian but non-Gaussian Y_i, splitting $d = d_Y + d_Q$ and bounding $\mathbb{P}(Y_i > d_Y) \leq e^{-(d_Y - m_i^+)^2/(2\sigma_i^2)}$ and $\mathbb{P}(Q_i > d_Q) \leq e^{-\theta_i^* d_Q}$ separately gives a slightly looser bound.

Kingman's bound to the execution times of s_i against this *minimum spacing* gives a second decay rate,

$$\theta_i^{\mathrm{s}} = \frac{2\left(c_i\, m_{i-1}^- - m_i^+\right)}{\sigma_{\mathrm{al},i}^2 + c_i\, \sigma_{\mathrm{al},i-1}^2}, \text{ with} \tag{10}$$

$$m_{i-1}^- = m_{i-1} - z_\gamma\, \sigma_{\mathrm{ep},i-1}, \tag{11}$$

which is positive only if $c_i\, m_{i-1}^- > m_i^+$, i.e., if s_{i-1} is slower even with its *optimistic* mean m_{i-1}^- (a faster predecessor spaces the items less). A predecessor with several workers can release items out of order and almost simultaneously; then, $\theta_i^{\mathrm{s}} = 0$. Both decay rates bound the same waiting time, so Eq. (9) uses the faster one, $\max(\theta_i^*, \theta_i^{\mathrm{s}})$. Eq. (10) treats the execution times of consecutive items at the two services as independent.

Implementation. Pegasus implements Poisson arrivals, gamma-distributed gaps with any C_a^2, where $\mathbb{E}[e^{-\theta T}] = (1 + \theta C_a^2/\lambda)^{-1/C_a^2}$, and periodic arrivals with uniform jitter; further renewal processes only need their Laplace transform. θ_i^* is the largest value that satisfies Eq. (7) on a logarithmic grid of 4000 values between 0.1 and $10^5\ \mathrm{s}^{-1}$; any smaller θ keeps the bound valid, so the grid can only loosen it. $d_i(\epsilon)$ is found by bisection on Eq. (9), returning the upper end of the final interval. For the log-scale GPs of the testbed, the shifts of Eqs. (6) and (11) are relative, $m_i^{\pm} = m_i\, e^{\pm z_\gamma \sigma_{\mathrm{ep},i}}$, and the predicted spread enters without a margin of its own.

C. Composing Bounds for Unseen Applications

The third step composes the models of the individual services into the PPG of the application S. We provide two variants, which trade assumptions against tightness.

Composition per Service (*Pegasus Node*). The end-to-end latency is the sum of the latencies at the K services, $D_S = \sum_{i=1}^K W_i$. An item can only exceed the sum of the service bounds if it exceeds at least one of them. Hence, if every service is bounded at the violation probability ϵ/K,

$$d_S = \sum_{i=1}^{K} d_i\Big(\frac{\epsilon}{K}\Big) \quad \text{implies } (\Rightarrow) \tag{12}$$

$$\mathbb{P}\left(D_S > d_S\right) \le \sum_{i=1}^{K} \mathbb{P}\left(W_i > d_i\Big(\frac{\epsilon}{K}\Big)\right) \le \epsilon. \tag{13}$$

This union bound requires no knowledge of how the latencies at different services depend on each other, so it remains valid if an item that is slow at one service is also slow at the next (e.g., a large video frame), and although consecutive queues are coupled. Its price is that every service is bounded further in the tail as the application grows, and pays for its own queue as if it stood alone, apart from the spacing of Eq. (10).

Composition per Application (*Pegasus SNC*). Stochastic Network Calculus (SNC) [9] composes the *service* that the nodes offer instead of their bounds: the application is treated as one system that receives the external arrivals once. In the MGF domain, the end-to-end latency of a application of FIFO services obeys

$$\mathbb{P}\left(D_S > d\right) \le e^{-\theta d} \prod_{i=1}^{K} \frac{M_i(\theta)}{1 - \phi_i(\theta)}, \text{ with} \tag{14}$$

$$\phi_i(\theta) = M_i(\theta)\, \mathbb{E}\big[e^{-\theta T}\big]^{c_i} < 1, \tag{15}$$

for every $\theta > 0$, where each factor is the latency moment of one service: its execution-time MGF, inflated by the queue that the load $\phi_i(\theta)$ causes. Solving for the latency at the violation probability ϵ and choosing the best θ yields

$$d_S = \min_{\theta > 0}\ \frac{1}{\theta}\left[\sum_{i=1}^{K} \ln \frac{M_i(\theta)}{1 - \phi_i(\theta)} - \ln \epsilon\right]. \tag{16}$$

In contrast to Eq. (12), the violation probability is not split: the term $-\ln\epsilon$ is paid once for the whole application, which is the "pay bursts only once" property of network calculus. Moreover, Eq. (16) makes no assumption about the arrivals inside the application, so Assumption 6 is not needed. In return, all services share one θ, so one service close to saturation loosens the whole bound. Further, the product in Eq. (14) requires that the execution times of an item are independent between the services; Hölder's inequality removes this requirement by multiplying the variance of s_i with an exponent $p_i \ge 1$, $\sum_i 1/p_i = 1$, which we report as an ablation. Finally, the derivation assumes that items leave every service in arrival order, which only holds for single-worker services; for several workers, we use the factor c_i in Eq. (15) as an approximation and validate it in Section V, whereas *Pegasus Node* remains a guarantee.

Required Information. For *Pegasus Node*, each service contributes one number, $d_i(\epsilon/K)$, which depends only on its own model, the arrival process, and m_{i-1}^- and $\sigma_{\mathrm{al},i-1}$ of its predecessor. *Pegasus SNC* needs m_i^+, $\sigma_{\mathrm{al},i}$, and c_i of all services in one place, because θ is optimized jointly.

Resulting Guarantee. Both variants satisfy Eq. (4) whenever the pessimistic mean of every service lies above its true mean, which each service fulfills with confidence γ according to its model. All K services fulfill it simultaneously with confidence at least $1-K(1-\gamma)$; a per-service level of $1-(1-\gamma)/K$ would restore γ for the application, at the price of larger pessimistic means. We apply γ per service and report in Section V how often the resulting bounds cover the measured latency quantile of the application.

Implementation. Eq. (16) is minimized over the same θ grid, which can only loosen the bound. Both variants are closed-form expressions that need neither simulation nor sampling, and their cost grows linearly with K. For the Hölder ablation, the exponents are tuned per configuration and θ by a fixed-point iteration on $1/p_i \propto \sigma_{\mathrm{al},i}/\sqrt{1-\phi_i(\theta)}$. Both variants extend to fork-join DAGs, whose end-to-end latency is the maximum over the P leaf-to-root paths: each path is bounded as a application at ϵ/P (*Pegasus SNC*), or each of its services at $\epsilon/\sum_p K_p$, with K_p services on path p (*Pegasus Node*), and the DAG bound is the largest path bound.

V. Evaluation

We evaluate Pegasus through a series of eight experiments that show its accuracy and complexity in comparison to baseline methods. As a first step, we generate a set of applications (*CV*, *Audio*, *Lidar*), for which Pegasus

A. Experimental Setup

Service Generation. To evaluate generalization across structural dimensions, we evaluate unseen service configurations, unseen service compositions, and their combination. This requires an ample set of services and service compositions; for this, we combine an existing set of microservices and configuration interfaces from prior work [28] and augment it through LLM-based code generation.

Baselines. We compare *Pegasus* against five baseline methods. Like *Pegasus*, each baseline is trained only on observations (or benchmarks) of individual services, never on a run of the evaluated composition S. The ground truth is the measured $(1-\epsilon)$ quantile of the end-to-end latency across S.

- **No queueing (*GP Sum*, *DNN Sum*):** per service, a model predicts the execution time from the configuration; the end-to-end latency is the $(1-\epsilon)$ quantile of their sum. *GP Sum* reuses the *Pegasus*'s GPs, whereas *DNN Sum* uses a feed-forward network with the standard deviation of its training residuals. GP and neural-network regressors are common performance models of configurable systems [3], [11], [20], [21], [29]; summing their per-service predictions is the most direct way to compose their performance without data of the composition.
- **Queueing formula (*GP Queue*):** the same GP predictions enter the Allen–Cunneen approximation of the mean waiting time in a G/G/c queue [4], which reduces to Kingman's formula [18] for one worker, with an exponential waiting-time tail [16]; the per-service $(1-\epsilon)$ quantiles are added. *GP Queue* accounts for queueing, but it is an approximation, not a bound, and it uses GP predictions without a margin for epistemic uncertainty.
- **Learned quantiles (*Quantile Union*, *Conformal*):** the same GPs are used to predicts every service's $(1-\epsilon/K)$ latency quantile; in *Quantile Union*, the K predictions are added as a union bound. *Conformal* additionally multiplies each prediction by a split-conformal correction factor calibrated on held-out training configurations [31]. In contrast to *Pegasus*, both baselines need a load test per service and configuration, and assume that services see the same arrival process as during isolated training.

As a reference, an *Oracle* adds the *measured* $(1-\epsilon/K)$ latency quantiles of the services inside the composition. The *Oracle* uses the ground truth and is not a competitor; it shows the tightest result any union bound over services can reach.

B. Performance Metrics

Guarantee Tightness. abcd

Computational Complexity. abcd

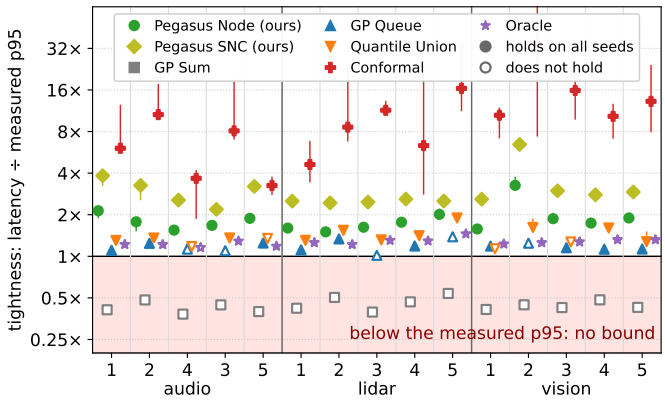


Fig. 3. Experiment B1 shows the tightness of the methods when trained on ...

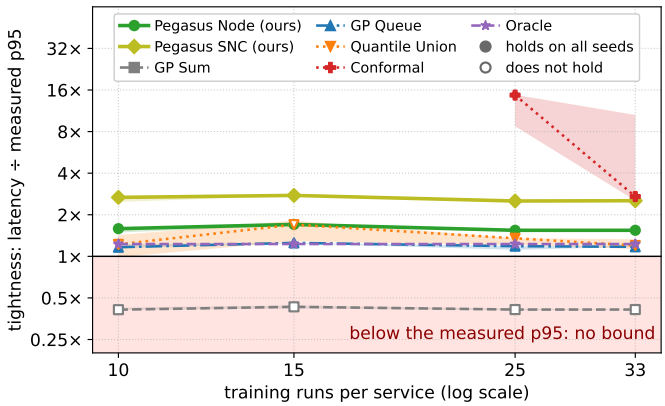


Fig. 4. Experiment B2 shows the sensitivity of the GPs to more/less training runs (The current version uses real data, but the problem is that we need longer runs, which we will get soon)

C. Coverage and Tightness of Service Bounds

1) Base Case: This will use most of the baselines from above to evaluate the maturity of our ideas vs. existing techniques.

a) Result: Fig. 3 shows...

2) Varying Evidence per Service: This makes use of the presented sampling strategies. The services are executed somewhere under an arbitrary service composition that is different from the target composition. We gradually train the PPGs with more of this data and see if this has an impact on the quality of the unseen target composition.

a) Result: Fig. 4 shows...

3) Varying Chain Depth: We evaluate how the validity, empirical coverage rate, and tightness of the derived PPGs scale with increasing depth of a map-reduce application. By evaluating synthetic and real chains across varying lengths $|S| \in \{1, 2, 5, 10, 20, 50\}$, we assess whether the composed bounds remain sufficiently tight and actionable for downstream optimization without having their bounds degraded.

Fig. 5 shows...

4) Varying Load and Arrival Pattern: Here we will use different arrival pattern (e.g., periodic, random (= Poisson), and batching) and different arrival rates (e.g., diurnal or bursty

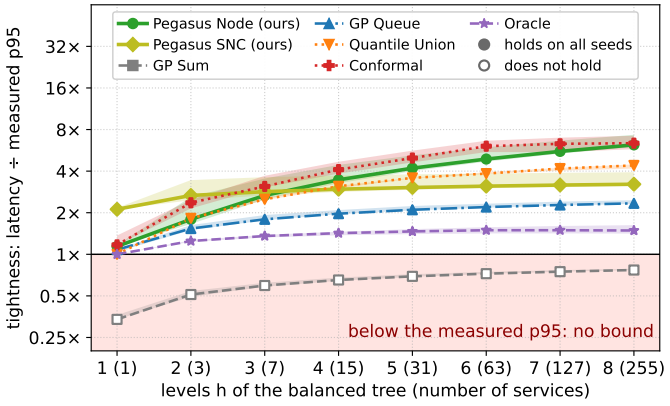
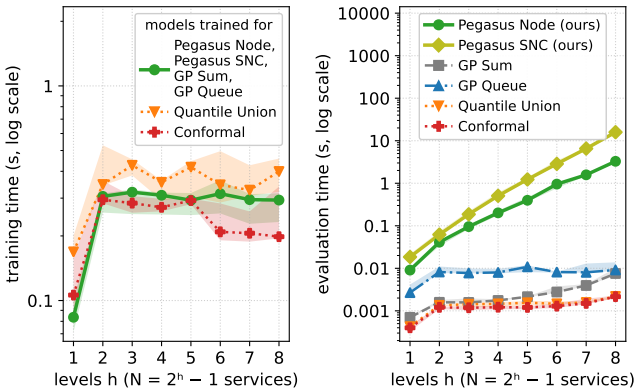


Fig. 5. Experiment B3 shows the scalability of ...



(a) abcd. (b) abcd.

Fig. 6. abcd:

with custom range). I already explored this once and found that we can characterize each of these arrival processes through the squared coefficient of variation (C_a^2) as defined in Eq. (2)

a) Result: Fig. 7a shows...

Fig. 7b shows...

5) Varying Tail Behaviors: Here I can look at services that have a different tail behavior (e.g., increasingly heavy/light tails) and how this impacts the quality of the guarantee.

a) Result: Fig. 8 shows...

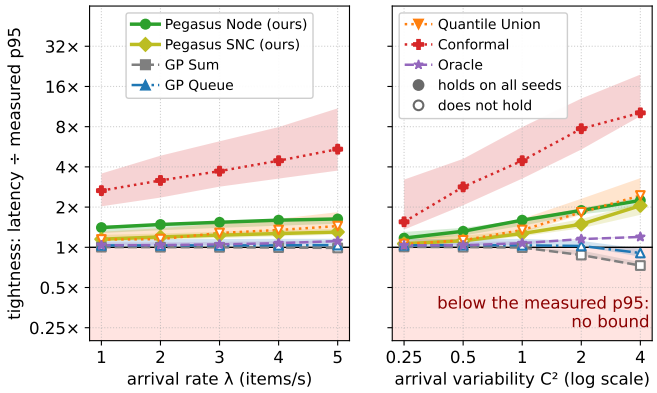
6) Varying Percentiles: Optionally, I could also look into different percentile guarantees and show where each method is more/less accurate.

a) Result: Fig. 9 shows...

D. Iterative Optimization of Service Bounds

I would integrate this into a use case, where we have to compose increasingly better applications using this method.

1) Fine-Tuning Bounds with E2E Samples: The aim of Pegasus is to provide performance guarantees for an application *before* benchmarking it end-to-end. To that extent, experiment C1 analyzes the utility of Pegasus as soon as end-to-end samples are available. To that extent we compare it with *Fluxion*, a method developed by Liang et al. [21] for predicting the application performance (P_{90} latency) from a composite GP model.



(a) abcd. (b) abcd.

Fig. 7. abcd: Experiment B4(a) shows the sensitivity of the methods to different arrival rates (λ, items per second); (the current version uses synthetic data, but nevertheless the utilizazion was set way too low, like what happens if we go to $\rho = 1.0$ for the busiest service?); Experiment B4(b) shows the sensitivity of the methods to different variability rates in the arrival process (C_a^2); (the current version uses synthetic data and I think the trend is clear: more variability makes the bound wider.)

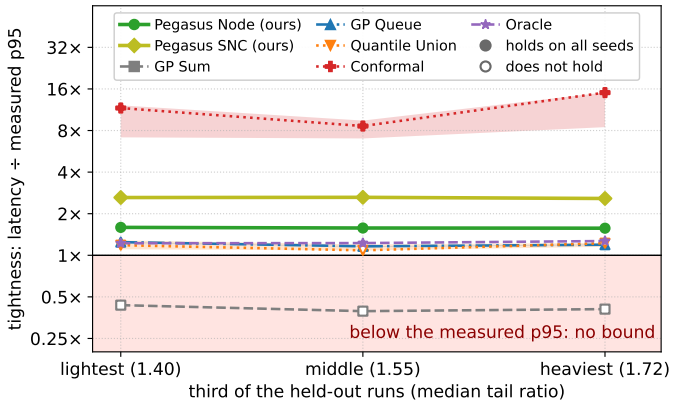


Fig. 8. Experiment B5 shows the sensitivity of the methods to different tail weights. (The current version uses real data where we could not detect a lot of variability in heavy-tailedness)

TODO: The discussion with Milos also showed that here we need to make (or restate) the argument that in comparison to Fluxion, we do not have to retrain if the topology changes. This can be done analytically.

a) Result: Fig. 10 shows...

2) Inferring Faster Service Compositions: This can go into two directions, first, we can optimize the service configuration according to the model, and second, we can improve the service composition.

For the first one, we can take *Fluxion* [21] (cf. Section 5.1.3) as inspiration, where they also used the model paired with an external optimizer to show how their model enables better performance optimization. They simply compared the different guarantee models (theirs, a plain GP, and a DNN) for coupled with different optimizers; this we can do as well, and it is also a nice cut between the guarantee (our contribution) and the external optimizer.

For the second one, it might be a bit more complex

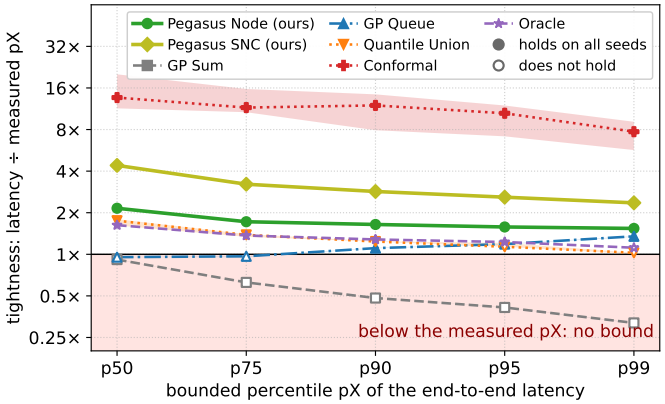


Fig. 9. Experiment B6 shows the sensitivity of the methods to different percentile levels used for bounding the latency. (The current version uses real data and I think this will be sufficient)

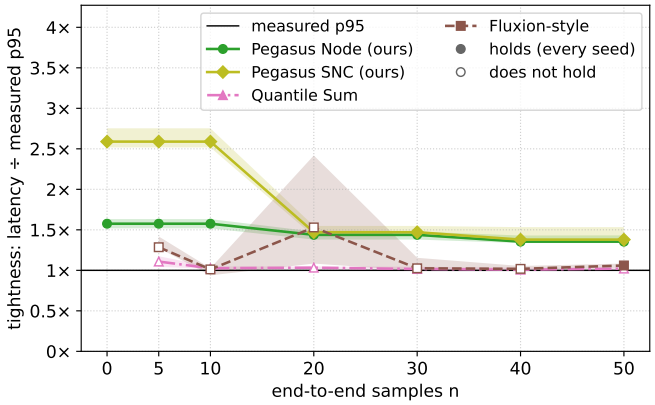


Fig. 10. Experiment C1 shows that...

because we have not talked a lot about the composition mechanism, but I would design it like the first one: we assume some composition mechanism that is external to us and we gradually show that by using our guarantees it can create better service composition. I think that this is the real deal because it repeatedly looks into unseen configurations, but it also the most complex evaluation. In any case, I think that a genetic algorithm could be the best choice here, because it builds a population of potential solutions and then iteratively improves them through sampling. But we do NOT have to include multiple rounds of benchmarking, but only one. The distinction would be:

- **single round**: we build a population of compositions and grade them according to different guarantees (ours vs. baselines). We repeat this step in a genetic algorithm until we have a solution for every guarantee. Then we benchmark these solutions and look at their fitness.
- **multiple round**: as above, but we use the empirical samples we captured from the benchmark and run the genetic algorithm (planning) and benchmarking again and again until we converge to a satisfying result. This we keep for future work at NSDI hehe

a) Result: Fig. 11 shows...

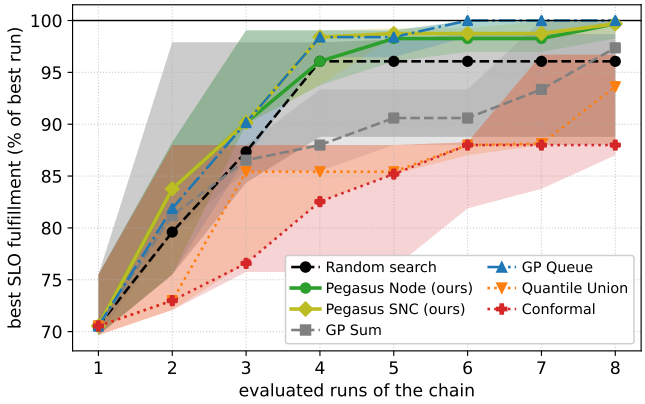


Fig. 11. Experiment C2 shows that...

VI. Discussion

@Víctor most likely

A. Limitations

Heavy-Tailed Execution Times. The performance of distributed systems is known to show heavy-tailed behavior, e.g., during garbage collection. *Pegasus* requires sub-Gaussian execution times (Assumption 4). If a Gaussian is fitted to a service with a heavier tail, the bound stays accurate where queueing dominates the latency, because a long queue depends mainly on the mean and the variance of the execution time; it becomes optimistic where the tail of a single execution dominates, i.e., at low utilization, for very regular arrivals, and for high percentiles. The per-service composition is affected more strongly in deep pipelines, as it bounds every service at ϵ/K.

Two extensions keep the framework applicable. First, services in production enforce timeouts; an execution time that is cut off at a timeout τ is bounded and therefore sub-Gaussian, so the guarantee holds for the service as deployed.

TODO: Draft of the mechanism: with a hard timeout, $Y_i \leq \tau$ always, and Bennett's inequality bounds the MGF from the variance and the range, which is tighter than a Gaussian with the range as scale. Without a hard timeout: cap at a high quantile and reserve a part of ϵ for items beyond the cap; this needs a proof for the items in the same busy period.

Second, network calculus has been extended to heavy-tailed processes with power-law instead of exponential envelopes [22]; the resulting bounds decay polynomially, are considerably looser, and lose the closed forms that make *Pegasus* fast to evaluate, which is why we do not adopt them.

Sampling from Evaluation Rounds. The idea was that we can drastically simplify experiments by not propagating decisions (e.g., *where* to explore) back to the services, but instead use the grading function to sample from all conducted runs. Given the coverage of the solution space, we can bound the inaccuracy this creates by finding the largest distance between two points, or a point and the parameter bounds.

VII. Conclusion & Future Work

@Víctor most likely

References

[1] Magrans de Abril, I., Sedlak, B., Pujol, V.C., Dustdar, S.: Composable Autonomic Offerings for Decentralized Service Provisioning in the Computing Continuum. In: Proceedings of the Workshop on Advanced Tools, Programming Languages, and PLatforms for Implementing and Evaluating algorithms for Distributed systems. pp. 1–9. ApPLIED'26 (Jul 2026). doi:10.1145/3820355.3820395

[2] Aiello, M.: A Paradigm Shift in Service Research: The Case of Service Composition. IEEE Transactions on Services Computing **18**(3), 1213–1215 (May 2025). doi:10.1109/TSC.2025.3552345

[3] Alipourfard, O., Liu, H.H., Chen, J., Venkataraman, S., Yu, M., Zhang, M.: CherryPick: Adaptively Unearthing the Best Cloud Configurations for Big Data Analytics. In: 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17). pp. 469–482 (2017)

[4] Allen, A.O.: Probability, statistics, and queueing theory with computer science applications. Academic Press Professional, Inc. (Aug 1990)

[5] Duan, Q.: Modeling and performance analysis for composite network–compute service provisioning in software-defined cloud environments. Digital Communications and Networks **1**(3), 181–190 (Aug 2015). doi:10.1016/j.dcan.2015.05.003

[6] Esashi, A., Lertpongrujikorn, P., Kato, S., Salehi, M.A.: Action Engine: Automatic Workflow Generation in FaaS. Future Generation Computer Systems **174**, 107947 (Jan 2026). doi:10.1016/j.future.2025.107947

[7] Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., Zhang, J.M.: Large Language Models for Software Engineering: Survey and Open Problems. In: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE). pp. 31–53 (May 2023). doi:10.1109/ICSE-FoSE59343.2023.00008

[8] Fanjiang, Y.Y., Syu, Y., Kuo, J.Y.: Search based approach to forecasting QoS attributes of web services using genetic programming. Information and Software Technology **80**, 158–174 (Dec 2016). doi:10.1016/j.infsof.2016.08.009

[9] Fidler, M., Rizk, A.: A Guide to the Stochastic Network Calculus. IEEE Communications Surveys & Tutorials **17**(1), 92–105 (2015). doi:10.1109/COMST.2014.2337060

[10] Franks, G., Al-Omari, T., Woodside, M., Das, O., Derisavi, S.: Enhanced Modeling and Solution of Layered Queueing Networks. IEEE Transactions on Software Engineering **35**(2), 148–161 (Mar 2009). doi:10.1109/TSE.2008.74

[11] Fu, S., Gupta, S., Mittal, R., Ratnasamy, S.: On the Use of ML for Blackbox System Performance Prediction. pp. 763–784 (2021)

[12] Gallotti, S., Ghezzi, C., Mirandola, R., Tamburrelli, G.: Quality Prediction of Service Compositions through Probabilistic Model Checking. In: Becker, S., Plasil, F., Reussner, R. (eds.) Quality of Software Architectures. Models and Architectures. pp. 119–134. Springer, Berlin, Heidelberg (2008). doi:10.1007/978-3-540-87879-7_8

[13] Gandhi, A., Harchol-Balter, M., Das, R., Lefurgy, C.: Optimal power allocation in server farms. In: Proceedings of the eleventh international joint conference on Measurement and modeling of computer systems. pp. 157–168. SIGMETRICS '09, Association for Computing Machinery, New York, NY, USA (Jun 2009). doi:10.1145/1555349.1555368

[14] Geebelen, D., Geebelen, K., Truyen, E., Michiels, S., Suykens, J.A.K., Vandewalle, J., Joosen, W.: QoS prediction for web service compositions using kernel-based quantile estimation with online adaptation of the constant offset. Information Sciences **268**, 397–424 (Jun 2014). doi:10.1016/j.ins.2013.12.063

[15] Guo, Y., Wu, M., Cao, Y., Wei, J., Ran, D., Yang, W., Xie, T.: BackendForge: Benchmarking Agentic End-to-End Code Generation with Backend Services (Jul 2026). doi:10.48550/arXiv.2607.11042, arXiv:2607.11042 [cs.SE]

[16] Harchol-Balter, M.: Performance modeling and design of computer systems: queueing theory in action. Cambridge University Press (2013)

[17] Iyer, R., Pedrosa, L., Zaostrovnykh, A., Pirelli, S., Argyraki, K., Candea, G.: Performance Contracts for Software Network Functions. In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19). pp. 517–530. USENIX Association (Feb 2019)

[18] Kingman, J.F.C.: The single server queue in heavy traffic. Mathematical Proceedings of the Cambridge Philosophical Society **57**(4), 902–904 (Oct 1961). doi:10.1017/S0305004100036094

[19] Kingman, J.F.C.: A martingale inequality in the theory of queues. Mathematical Proceedings of the Cambridge Philosophical Society **60**(2), 359–361 (Apr 1964). doi:10.1017/S0305004100037841

[20] Li, Z.L., Liang, C.J.M., He, W., Zhu, L., Dai, W., Jiang, J., Sun, G.: Metis: Robustly Tuning Tail Latencies of Cloud Systems. pp. 981–992 (2018)

[21] Liang, C.J.M., Fang, Z., Xie, Y., Yang, F., Li, Z.L., Zhang, L.L., Yang, M., Zhou, L.: On Modular Learning of Distributed Systems for Predicting End-to-End Latency. pp. 1081–1095 (2023)

[22] Liebeherr, J., View Profile, Burchard, A., View Profile, Ciucu, F., View Profile: Delay Bounds in Communication Networks With Heavy-Tailed and Self-Similar Traffic. IEEE Transactions on Information Theory **58**(2), 1010–1024 (Feb 2012). doi:10.1109/TIT.2011.2173713

[23] Marinescu, D.C.: Chapter 10 - Complex Systems and Self-Organization. In: Cloud Computing, pp. 301–315. Morgan Kaufmann, Boston (Jan 2013). doi:10.1016/B978-0-12-404627-6.00010-5

[24] Meflah, W., Brabra, H., Acheli, M., Douma, H.B., Sellami, M., Gaaloul, W., Zeghlache, D.: From Natural Language to TOSCA: Leveraging LLMs for Automated Service Composition. In: Aiello, M., Deng, S., Murillo, J.M., Georgievski, I., Benatallah, B., Wang, Z. (eds.) Service-Oriented Computing. pp. 3–18. Springer Nature, Singapore (2026). doi:10.1007/978-981-95-5012-8_1

[25] Papazoglou, M.P., Georgakopoulos, D.: Introduction: Service-oriented computing. Communications of the ACM **46**(10), 24–28 (Oct 2003). doi:10.1145/944217.944233

[26] Rzadca, K., Findeisen, P., Swiderski, J., Zych, P., Broniek, P., Kusmierek, J., Nowak, P., Strack, B., Witusowski, P., Hand, S., Wilkes, J.: Autopilot: workload autoscaling at Google. In: Proceedings of the Fifteenth European Conference on Computer Systems. pp. 1–16. EuroSys '20, New York, NY, USA (Apr 2020). doi:10.1145/3342195.3387524

[27] Sedlak, B., Pujol, V.C., Motlagh, N.H., Morabito, R., Wilhelmi, F., de Abril, I.M., Murturi, I., Bellalta, B., Donta, P.K., Dustdar, S.: Large-Scale Sensor Systems for Sentient Environments: Emerging Research Directions. IEEE Journal of Selected Areas in Sensors pp. 1–15 (2026). doi:10.1109/JSAS.2026.3716613

[28] Sedlak, B., Raith, P., Morichetta, A., Pujol, V.C., Dustdar, S.: Multi-Dimensional Autoscaling of Stream Processing Services on Edge Devices. IEEE Transactions on Services Computing pp. 1–13 (2026). doi:10.1109/TSC.2026.3711142

[29] Van Aken, D., Pavlo, A., Gordon, G.J., Zhang, B.: Automatic Database Management System Tuning Through Large-scale Machine Learning. In: Proceedings of the 2017 ACM International Conference on Management of Data. pp. 1009–1024. SIGMOD '17, Association for Computing Machinery, New York, NY, USA (May 2017). doi:10.1145/3035918.3064029

[30] Venkataraman, S., Yang, Z., Franklin, M., Recht, B., Stoica, I.: Ernest: Efficient Performance Prediction for Large-Scale Advanced Analytics. In: 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16). pp. 363–378 (2016)

[31] Vovk, V., Petej, I., Toccaceli, P., Gammerman, A., Ahlberg, E., Carlsson, L.: Conformal calibration. In: Proceedings of Machine Learning Research, pp. 84–99 (Sep 2020)

[32] Wilhelm, R., Engblom, J., Ermedahl, A., et al.: The worst-case execution-time problem—overview of methods and survey of tools. ACM Transactions on Embedded Computing Systems **7**(3), 1–53 (2008)

[33] Wolff, R.W.: An Upper Bound for Multi-Channel Queues. Journal of Applied Probability **14**(4), 884–888 (1977). doi:10.2307/3213363

[34] Wu, Q., Zhang, M., Zheng, R., Lou, Y., Wei, W.: A QoS-Satisfied Prediction Model for Cloud-Service Composition Based on a Hidden Markov Model. Mathematical Problems in Engineering **2013**(1), 387083 (2013). doi:10.1155/2013/387083

[35] Yu, G., Chen, P., Zheng, Z.: Microscaler: Cost-Effective Scaling for Microservice Applications in the Cloud With an Online Learning Approach. IEEE Transactions on Cloud Computing **10**(2), 1100–1116 (Apr 2022). doi:10.1109/TCC.2020.2985352



Boris Sedlak is a postdoctoral researcher at the Distributed Systems Group at TU Wien, Austria, where he received his PhD in 2025. Prior to that, he received his B.Sc. in Media Informatics at the University of Applied Sciences in St. Pölten, and his M.Sc. in Software Engineering & Internet Computing at the TU Wien. His research interests include edge intelligence, causal methods for the computing continuum, and service-oriented computing.



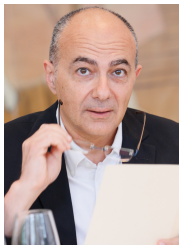
Milos Gravara ...



Víctor Casamayor Pujol is a tenure-track assistant professor at Universitat Pompeu Fabra (UPF), Barcelona, Spain. Prior to that, he worked as a postdoctoral researcher in the Distributed Systems Group at TU Vienna after having earned his PhD at UPF. His research interests revolve around self-adaptive methodologies for computing continuum systems, including SLO-based definitions, causal and machine learning inference, and robotics.

Stefan Nastic ...





Schahram Dustdar received his Ph.D. in business informatics from the University of Linz, Austria, in 1992. He is a full professor of computer science, focusing on internet technologies, and heads the Distributed Systems Group at TU Wien, Vienna, Austria. He is chairman of the Informatics Section of the Academia Europaea. He has been an associate editor for IEEE Transactions on Services Computing, ACM Transactions on the Web, and ACM Transactions on Internet Technology. He has received several accolades, including the ACM Distinguished Scientist and IBM Faculty awards.

POTENTIAL APPENDIX

- List of the services and their states and configuration knobs in more detail. This can be clusters of service types, as for *Fluxion* in [21].
- Description of the code artifact, or at least a pointer to the extended description in the README there.